An Architectural Reference for Enterprise AI Agents, Fixed-Seat Subscriptions, and Telemetry Integration

Framework: 3-Tier Evidence Classification | Scope: M365 Copilot, Copilot Studio, Azure AI Foundry, Fabric, GitHub | Target: Technical & Executive Stakeholders

An Architectural Reference for Enterprise AI Agents, Fixed-Seat Subscriptions, and Telemetry Integration

Executive Overview & Evidence Framework

This document provides a research-backed reference on how telemetry, usage data, and seat licenses operate across the Microsoft enterprise ecosystem.

To ensure architectural integrity during technical and executive reviews, every metric in this guide is categorized into a **3-Tier Evidence Framework**:
* **Tier 1 — Raw Documented Platform Telemetry**: Exact properties returned directly by Microsoft and GitHub APIs or database schemas.
* **Tier 2 — Backend Derived Metrics**: Quantitative indicators calculated by our ingestion pipelines from Tier 1 data (e.g., Code Completion Acceptance Rate, Activity Ratios).
* **Tier 3 — Business Estimates & Inferred Outcomes**: Business-layer calculations requiring external baselines, rate cards, or policy thresholds (e.g., Estimated Financial Savings, Potential License Optimization).

1. The 4 Microsoft Agent Pathways (APIs, Schemas & Admin Setup)

Pathway 1: Declarative / No-Code Agents (Microsoft 365 Copilot)

Configuration-driven assistants scoped to SharePoint, OneDrive, and Teams data defined via `declarativeAgent.json`.

1. Architectural Reality & Telemetry Scope

- Declarative agents run inside the Microsoft 365 Copilot execution boundary.
- **Critical Scope Boundary**: The Microsoft Graph usage reporting API is an **employee adoption report** for licensed Microsoft 365 Copilot users across Microsoft 365 host applications. It is **not** a per-agent execution trace API. It tracks whether employees are actively using Copilot within Word, Excel, Teams, etc., and records a `copilotAgentLastActivityDate` in Version 2. It does not output conversation transcripts, prompt text, or raw per-agent token counts.

2. Exact API Endpoint & Authentication

- **Documented Graph Endpoint**:
  `http GET https://graph.microsoft.com/v1.0/copilot/reports/getMicrosoft365CopilotUsageUserDetail(period='D28')`
  (Note: Microsoft supports periods `'D7'`, `'D28'`, `'D90'`, *and* `'D180'` *under Version 2; Version 1 supports* `'D30'`*).*
- **Response Delivery**: Returns a downloadable CSV by default, or JSON when requested with `$format=application/json`.
- **Required Permission**: `Reports.Read.All` (Application permission requiring Microsoft Entra Global Admin consent).
- **Documented Admin Roles**: Global Administrator, AI Administrator, Exchange Administrator, SharePoint Administrator, Teams Administrator, or Reports Reader.

3. Data Schema & Metrics Classification

- **Tier 1 (Raw API Properties)**:
- `reportRefreshDate`: Date when the report dataset was refreshed (e.g., `2026-10-24`).
- `userPrincipalName`: Employee email identifier (e.g., `employee@company.com`).
- `displayName`: Full employee name.
- `lastActivityDate`: Date of the employee's most recent interaction with any M365 Copilot feature.
- Application-Specific Activity Dates: `copilotChatLastActivityDate`, `microsoftTeamsCopilotLastActivityDate`, `wordCopilotLastActivityDate`, `excelCopilotLastActivityDate`, `powerPointCopilotLastActivityDate`, `outlookCopilotLastActivityDate`, `oneNoteCopilotLastActivityDate`, `loopCopilotLastActivityDate`.
- `copilotAgentLastActivityDate` *(Report v2)*: Timestamp of last interaction with any declarative agent.
- **Tier 2 (Derived Dashboard Metrics)**:
- `Active vs. Inactive Licensed Users`: Calculated by evaluating `lastActivityDate` against a chosen observation window (e.g., 14, 28, or 60 days).
- `Application Adoption Breakdown`: Percentage of licensed users active in Teams Copilot vs. Excel Copilot vs. Word Copilot.
- **Privacy & Tenant Masking Caveat**:
- If the organization has enabled "*Conceal user, group, and site names in all reports*" under Microsoft 365 Admin Center Org Settings, `userPrincipalName` and `displayName` are returned as anonymized hash strings. The Microsoft 365 Global Admin must disable this setting to display authentic employee emails.

Pathway 2: Low-Code Workflow Agents (Microsoft Copilot Studio)

Autonomous chatbots and business workflow agents built on Power Platform and Microsoft Dataverse.

1. Architectural Reality & Telemetry Scope

- Copilot Studio conversations are logged automatically into Microsoft Dataverse inside the `conversationtranscript` table.
- **Critical Scope Boundary**: Dataverse transcript records store raw message exchanges and event activities. Business metrics such as resolution status, escalation events, and satisfaction scores **are not top-level table columns**; they must be parsed from the JSON activities inside the `content` field or pulled from session analytics exports.

2. Exact API Endpoint & Authentication

- **Documented Dataverse Web API Query**:
  `http GET https://<org-env>.api.crm.dynamics.com/api/data/v9.2/conversationtranscripts?$select=conversationtranscriptid,name,conversationstarttime,schematype,schemaversion,content,metadata,_bot_conversationtranscriptid_value`
- **Required Security Role**: `Bot Transcript Viewer` or `System Administrator` in the Power Platform environment.
- **Dataverse Ingestion & Retention Rules**:
- By default, a Dataverse bulk deletion job removes transcripts older than 30 days unless customized by the administrator.
- Transcripts larger than 1 MB are split across multiple records linked by `BatchId` in `metadata`.
- Transcripts are not logged for developer environments or Dataverse for Teams.

3. Data Schema & Metrics Classification

- **Tier 1 (Raw Dataverse Entity Columns)**:
- `conversationtranscriptid`: Primary key GUID for the transcript record.
- `name`: Transcript record title / session identifier.
- `conversationstarttime`: UTC start timestamp of the conversation.
- `_bot_conversationtranscriptid_value`: Web API lookup value referencing the Agent GUID in the `bot` entity.
- `content`: Memo/text field holding the full conversation activity stream in JSON format.
- `metadata`: JSON payload containing Agent ID, Tenant ID, Agent Name, and Batch ID.
- **Tier 2 (Parsed / Derived Analytics)**:
- `Total Conversation Sessions`: Distinct session count aggregated over the reporting timeframe.
- `Session Outcomes`: Derived by parsing the `content` JSON for outcome events:
  - `Resolved`: Bot completed the workflow without human escalation.
  - `Escalated`: Conversation transferred to a human agent via Omnichannel / ServiceNow.
  - `Abandon`: User exited the conversation without completion.
- `Resolution Rate (%)`: `(Resolved Sessions / Total Evaluated Sessions) * 100`.
- `Customer Satisfaction (CSAT)`: Extracted from `content` only when a CSAT survey topic is explicitly triggered and answered by the user.
- **Identity Attribution Note**:
- For internal agents authenticated via Teams or SharePoint with SSO, user identifiers are logged in the activity stream. However, depending on channel configuration, user IDs may be pseudonymized or hashed. Plaintext employee emails require explicit identity propagation in bot variables.

Pathway 3: Code-First Custom Agents (Azure AI Foundry / OpenAI / App Insights)

Full-stack software agents developed by software engineers in Python or C# deployed to Azure cloud infrastructure.

1. Architectural Reality & Observability Control

- Custom agents communicate directly with Azure OpenAI model deployments and log execution traces into Azure Application Insights.
- This pathway provides the **highest level of technical observability**, because engineering controls both the inference interface and application telemetry baggage.

2. Exact API Endpoints & Schemas

1. **Azure OpenAI Chat Completions API** (Direct Model Telemetry):
2. Documented usage payload returned on every completion call:
   `json "usage": { "prompt_tokens": 128, "completion_tokens": 85, "total_tokens": 213 }`
   (Note: The newer Responses API utilizes `input_tokens` and `output_tokens`).
3. **Azure Application Insights REST API** (Logs Query v1):
4. **Endpoint**:
   `http POST https://api.applicationinsights.io/v1/apps/{appId}/query`
5. **Schema Context**: Queries tabular telemetry across `requests` (inbound triggers), `dependencies` (outbound LLM/API calls), and `traces`.
6. `customDimensions` is a dynamic column on these tables used to store application-injected attributes.

7. **Azure Cost Management Query API**:
8. **Endpoint**:
   `http POST https://management.azure.com/subscriptions/{subId}/providers/Microsoft.CostManagement/query?api-version=2023-11-01`
9. **Schema Context**: Returns aggregated billing datasets (`PreTaxCost` , `ResourceGroup` , `Currency`). Cost Management reports billing aggregated by resource, meter, or tag; it does not provide an automatic per-prompt invoice line item.

3. Data Schema & Metrics Classification

- **Tier 1 (Observed Platform Telemetry)**:
- Model Token Usage: `prompt_tokens` , `completion_tokens` , `total_tokens` .
- Latency & Execution Health: `duration` , `success` , `resultCode` from `requests` and `dependencies` .
- **Tier 2 (Application-Instrumented Properties)**:
- `user_email` : Authenticated employee email injected into OpenTelemetry baggage by our code.
- `task_category` : Business category emitted by application code (e.g., `Hardware_Triage` , `Access_Request`).
- `Tool Call Latency` : Latency of third-party API dependencies (e.g., ServiceNow REST calls).
- **Tier 3 (Calculated Financial & Operational Estimates)**:
- `Estimated Model Cost ($)` : Calculated by multiplying observed token counts by Azure rate cards:
  $$ ext{Estimated Cost} = \left(rac{ ext{prompt_tokens} imes ext{Input Rate}}{1,000,000}
  ight) + \left(rac{ ext{completion_tokens} imes ext{Output Rate}}{1,000,000}
  ight)$$
- `Estimated Time Saved` : Calculated by subtracting automated latency from an approved manual baseline duration (e.g., standard 25-minute IT triage baseline).

Pathway 4: Governed Data Agents (Microsoft Fabric Data Agents)

Specialized analytics agents built over Fabric OneLake, Lakehouses, and Power BI semantic models.

1. Architectural Reality & Multi-Source Telemetry

- Fabric Data Agents query enterprise data estates. Monitoring requires separating **audit events**, **workspace topology**, and **compute capacity consumption** across distinct sources.

2. Exact API Sources & Functions

1. **Power BI / Fabric Activity Events API (Audit Log)**:
2. **Endpoint**:
   `http GET https://api.powerbi.com/v1.0/myorg/admin/activityevents?startDateTime='...'&endDateTime='...'`
3. **Function**: An audit trail of operations executed across the tenant.
4. **Documented Fields**: `Activity` (operation name), `UserId` (user UPN), `WorkSpaceName` , `ArtifactName` , `CapacityId` , `CapacityName` , `IsSuccess` .
5. **Authentication Caveat**: Requires Fabric Administrator role. When authenticated as a Service Principal, admin-consent Power BI permissions must not be configured in Azure Portal.
6. **Fabric Workspace Metadata API**:
7. **Endpoint**:
   `http GET https://api.fabric.microsoft.com/v1/workspaces/{workspaceId}`
8. **Function**: Returns workspace configuration (`id` , `displayName` , `capacityId`). It does **not** return Capacity Unit (CU) consumption or query duration.
9. **Fabric Capacity Consumption Telemetry**:
10. **Source**: Microsoft Fabric Capacity Metrics App and Fabric capacity usage event streams.
11. **Data Agent Operation**: Documented as `Operation: "AI Query"` , `Item Kind: "LlmPlugin"` .
12. **Compute Metric**: Capacity Units (`CU Seconds`) billed against Fabric capacity based on input, cached input, and output processing rates.

---

2. Enterprise AI Subscriptions (Fixed Seats & Developer Tools)

Enterprises license developer and knowledge-worker AI tools primarily through fixed-seat monthly subscriptions:

Subscription Tier	Enterprise List Price*	Primary Telemetry Source	Core Measurable Telemetry
GitHub Copilot Business	**$19 / user / mo**	GitHub REST API (`/billing/seats` & `/copilot/metrics/ ...`)	Seat assignment dates, last active editor, code completions, acceptance actions, language breakdowns.
GitHub Copilot Enterprise	**$39 / user / mo**	GitHub REST API (`/enterprises/{ent}/copilot/ ...`)	Everything in Business + PR summaries, organization-wide knowledge base indexing, custom code policies.
Microsoft 365 Copilot	**$30 / user / mo**	Microsoft Graph Reports API (`/copilot/reports/ ...`)	Active vs. Inactive users, application breakdown (Teams, Word, Excel, Outlook), last active date per employee.
Microsoft Copilot Studio	**$200 / tenant / mo** *(25k msgs)*	Dataverse OData Web API (`conversationtranscripts`)	Total sessions, parsed resolution/escalation counts, CSAT survey responses, complete conversation logs.

**Pricing Note: Prices represent standard US commercial list pricing. Actual enterprise billing is governed by specific enterprise agreements (EA), regional tiers, and volume licensing discounts.*

GitHub Copilot Telemetry: Raw Schemas vs. Derived Metrics

GitHub separates seat management from developer code interaction across two distinct API surfaces:

1. GitHub Copilot Seats API (License & Inactivity Status)

- **Endpoint**: `GET https://api.github.com/orgs/{org}/copilot/billing/seats`
- **Pagination**: Default 50 records per page (maximum 100).
- **Tier 1 (Raw API Schema Fields)**:
- `total_seats`: Total number of provisioned/billed licenses assigned to the organization.
- `seats[].assignee.login`: Developer's GitHub username.
- `seats[].assignee.id`: Developer's GitHub numeric ID.
- `seats[].created_at`: Timestamp when the license was assigned.
- `seats[].last_activity_at`: Timestamp of the developer's last active Copilot trigger.
- `seats[].last_activity_editor`: Editor and extension version string (e.g., `vscode/1.94.0/copilot/1.240.0`).
- `seats[].plan_type`: Assigned tier (`business` or `enterprise`).
- **Telemetry Caveat**: `last_activity_at` relies on IDE telemetry being enabled by the developer. If a developer disables telemetry in their IDE, their activity timestamp will not update.

2. GitHub Copilot Usage Metrics API (Code Interaction Activity)

- **Endpoint**: `GET https://api.github.com/orgs/{org}/copilot/metrics/reports/users-28-day/latest`
- **Delivery Mechanism**: Returns signed URLs to download daily or 28-day newline-delimited JSON (`.ndjson`) files.
- **Tier 1 (Raw Fields Inside Documented NDJSON Report)**:
- `day`: Reporting activity date (`YYYY-MM-DD`).
- `user_login`: Developer username.
- `code_generation_activity_count`: Number of distinct Copilot suggestion generation events.
- `code_acceptance_activity_count`: Number of accepted suggestion events.
- `loc_suggested_to_add_sum`: Total lines of code suggested to add (excludes agent edits).
- `loc_added_sum`: Total lines of code added to the editor (includes agent/edit-mode changes).
- `totals_by_feature`: Array breaking down activity by surface (e.g., `code_completion`, `copilot_app`).
- `totals_by_language_feature`: Array breaking down activity by programming language.

3. How Derived Dashboard Metrics Are Calculated

Dashboard Metric	Formula & Scope	Classification	Strategic Purpose
Code Completion Acceptance Rate (%)	$$\left(\rac{\sum ext{code_acceptance_activity_count (for code_completion)}}{\sum ext{code_generation_activity_count (for code_completion)}}		
ight) imes 100$$	**Tier 2 (Derived)**	Evaluates suggestion relevance specifically for inline completions, preventing distortion from chat or agent-mode events.	
Added-to-Suggested Lines Ratio (%)	$$\left(\rac{ ext{loc_added_sum}}{ ext{loc_suggested_to_add_sum}}		
ight) imes 100$$	**Tier 2 (Derived)**	Tracks editor code additions relative to completions. *(Note: Not a measure of permanent code retention).*	
Potential Inactive License Savings ($)	If $(ext{Current Date} - ext{last_activity_at}) > 30 ext{ days}$: $$ ext{Potential Savings} = ext{Count}(ext{Inactive Seats}) imes ext{Seat Cost}$$	**Tier 3 (Estimate)**	Flags potentially abandoned seats for internal review and license reclamation.

3. Practical Implementation Strategy With Our Current Resources

> **Architecture & Ingestion Note**: Telemetry across all four pathways arrives in different formats (Graph CSV/JSON, Dataverse OData, App Insights KQL, and GitHub NDJSON). Our backend data pipelines extract, normalize, and load these inputs into standardized database schemas to power the executive dashboard.

3.1 Fixed Seats Telemetry: What We Need From Balaji (Admin)

To ingest authentic employee adoption and identify potential inactive license savings, we only need from Balaji:
* **Option A: Microsoft 365 Copilot Usage CSV Export** from the M365 Admin Center (`admin.microsoft.com → Reports → Usage → Microsoft 365 Copilot → Export`), **OR**
* **Option B: Azure Entra App Registration credentials** (`Tenant ID` , `Client ID` , `Client Secret`) with `Reports.Read.All` application permission.

This provides the authentic list of licensed company employees, active usage dates, and inactive seats to quantify immediate cost-optimization savings.

3.2 Custom Autonomous Agents: Live Demonstration via Azure AI Foundry

We demonstrate code-first agent observability in real time using our dedicated Azure OpenAI model deployment:

1. **Live Employee Query Execution**:
2. The employee interacts with the IT Support Agent to resolve real workplace requests (e.g., device triage, software setup).
3. The query is processed live by our Azure OpenAI `gpt-4o` deployment in `claimsGuru` resource group.
4. **Observed & Calculated Telemetry Pipeline**:
5. **Observed Platform Telemetry (Tier 1)**: Exact prompt and completion token counts returned directly by the model inference response, plus request execution duration.
6. **Application-Instrumented Properties (Tier 2)**: Authenticated employee email (`user_email`) and task classification captured directly from the session.
7. **Estimated Financial & Operational Metrics (Tier 3)**:
   - *Estimated Inference Cost*: Calculated using Azure model rate cards ($2.50 / 1M input tokens, $10.00 / 1M output tokens).
   - *Estimated Time Saved*: Derived by comparing automated handling duration against an approved manual IT baseline (e.g., 25-minute standard baseline).

4. How to Identify What Type of Agents Are in Your Stakeholder's GitHub Repo

When your stakeholder shares a GitHub repository containing AI agents, inspect its file tree to immediately determine its architecture:

```
Stakeholder GitHub Repository
|
├── CASE A: Declarative Agent (M365 Copilot Extension)
|   ├── appPackage/
|   |   ├── manifest.json
|   |   └── declarativeAgent.json   <-- Look for this manifest file!
|   └── (No executable backend code; configuration package only)
|
├── CASE B: Copilot Studio Solution (Power Platform Unpack)
|   ├── Solution.xml
|   ├── bots/                       <-- Look for bots & botcomponents folders!
|   └── botcomponents/
|   └── (Exported low-code solution; runs in Dataverse)
|
└── CASE C: Code-First Custom Agent (Foundry / Semantic Kernel)
    ├── requirements.txt / pyproject.toml
    ├── semantic_kernel/ or azure.ai
    ├── plugins/
    └── main.py / agent.py          <-- Full Python/C# execution logic!
```

Action Steps Based on Repo Structure:

1. **If Case A (Declarative Agent)**:
2. Do not run the repo code. It is an M365 manifest package.
3. Ingest usage data via the **Microsoft Graph Reports API** (`/v1.0/copilot/reports/ ...`).
4. **If Case B (Copilot Studio Export)**:
5. It is a Power Platform solution.
6. Ingest session logs via the **Dataverse Web API** (`conversationtranscripts`).
7. **If Case C (Code-First Custom Agent)**:
8. It is custom software running on Azure.
9. Ingest telemetry via **Azure Application Insights** or configure a webhook in their agent event loop.
